



Node Groups, Node IAM, IP caps, Addons

Worker nodes are EC2 instances a managed node group runs for you. Each assumes a node role, Spot trades reliability for cost, and the instance type caps pods per node. Addons make the overall setup work within EKS.

IN THIS LECTURE

01 Managed node groups

EKS runs the EC2 instances for you.

02 Joining the cluster

How an instance becomes a registered, Ready node.

03 The node role

Join the cluster, run pod networking, pull images.

04 Spot capacity

The cost versus interruption tradeoff, and pool diversity.

05 Pod density

How interface slots and the networking mode cap pods per node.

06 Addons

EKS-managed versus self-managed core components.



Managed vs self-managed node groups

Both run the same EC2 instances, out of the same auto scaling group and launch template. The difference is who provisions, joins and upgrades them.



The instances

plain EC2, running kubelet



Auto scaling group

holds desired, min, max



Launch template

image, size, bootstrap

Self-managed

you operate all three

- ✓ You create the auto scaling group and the launch template.
- ✓ You own the bootstrap that joins each instance to the cluster.
- ! Every upgrade and replacement is yours to drive. Maximum control, maximum work.

Managed node group

this course

- ✓ You declare instance types, desired, min, max and subnets.
- ✓ EKS provisions the instances, joins them, and runs the scaling group and template underneath.
- ✓ Version upgrades and node replacements are drained gracefully by EKS.

A third option: EKS Auto Mode



AWS picks and provisions nodes straight from pending pods, so there is no node group to declare. It adds a management fee per instance and takes per-instance control away, so we stay with node groups here.

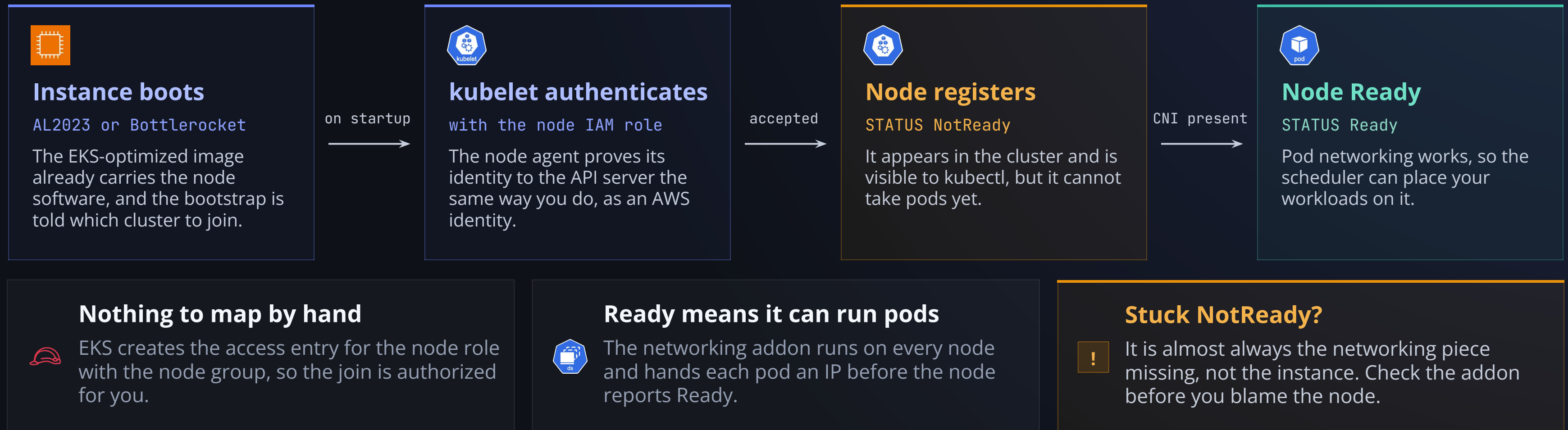
Managed is the default choice

- ✓ The same EC2 instances, far less to build and maintain. From here on, adding nodes means declaring a managed node group.

THE JOIN SEQUENCE

How a node joins the cluster

An instance boots from an EKS-optimized image, its kubelet authenticates with the node role, it registers, and it turns Ready only once pod networking is in place.



THE NODE ROLE

One role, three jobs: join, network, pull images

The same trust-plus-permissions pattern as the cluster role. Trusted by the EC2 service, and granted three categories of permission, one AWS managed policy each.



Node IAM role

```
"Service": "ec2.amazonaws.com"
```

Trusted by EC2, because the node is an EC2 instance. Every instance in the group assumes it.

grants →



Join and talk to the cluster

The kubelet registers the node, reads the EC2 details it is named from, and keeps talking to the control plane.

AmazonEKSWorkerNodePolicy



Run pod networking

Manage the network interfaces and IP addresses that pods receive.

AmazonEKS_CNI_Policy



Pull container images

Read images from the registry so your containers can start.

AmazonEC2ContainerRegistryPullOnly

Delivered by instance profile



The kubelet picks the credentials up from instance metadata. No keys are written to the node.

Not the cluster role



Two separate roles, two different trusted services. A node cannot reuse the role that created the cluster.

Create it once, reuse it



One node role can back every node group in the cluster, On-Demand and Spot alike.

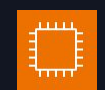
Spot vs On-Demand capacity

The same instances under two purchasing models. One costs more and is never reclaimed, the other is deeply discounted and can be.

70% Typical Spot saving, and up to 90% on some types.

2 min Warning before AWS reclaims a Spot instance.

0 Differences in the hardware you get. Same instances.



On-Demand

the default purchasing model

- ✓ Full hourly rate, billed per second while it runs.
- ✓ Yours until you release it, never reclaimed by AWS.
- ✓ The choice for workloads that cannot tolerate interruption.



Spot

spare capacity, discounted

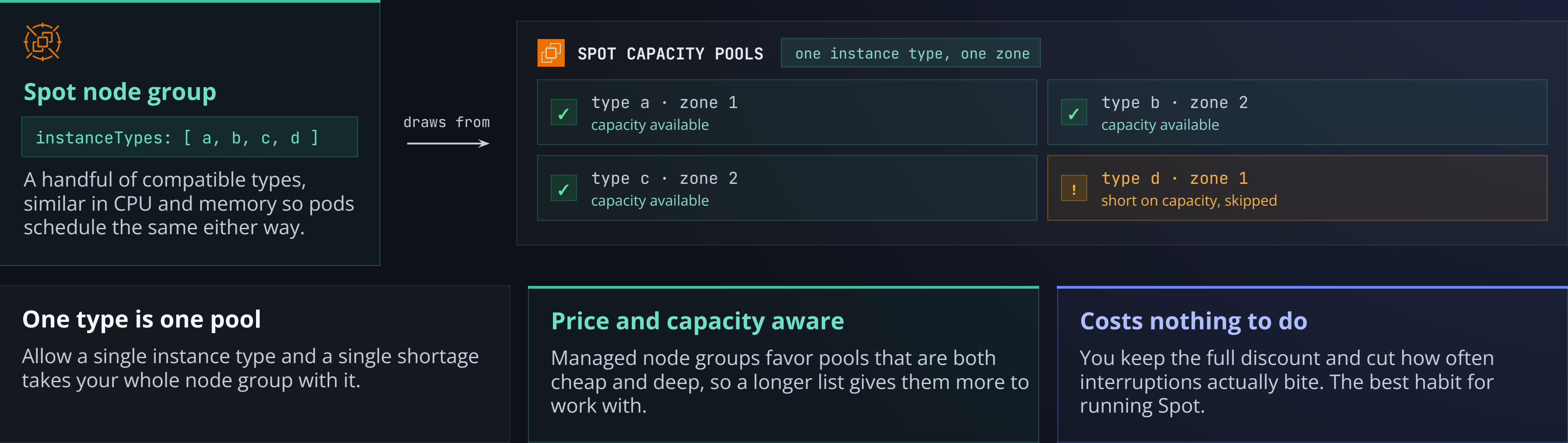
this course

- ✓ The same instance types at a fraction of the price.
- ! AWS can reclaim it with a two-minute notice, and the node group drains the node when that notice arrives.
- ✓ Ideal for labs and stateless workloads that reschedule elsewhere.

- ✓ **This course runs Spot**, because the savings outweigh an occasional interruption in a lab. Production blends the two: On-Demand node groups for what must stay up, Spot for what can move.

Diversify Spot across instance types

One instance type in one zone is one capacity pool. Listing several types lets the node group draw from whichever pool is healthy.



Interface slots cap how many pods a node can run

Every pod takes a real VPC address from a slot on the node’s network interfaces. The instance type fixes the slot count, and the networking mode decides whether a slot holds one address or sixteen.



$$\text{max pods} = (\text{interfaces} \times (\text{slots} - 1) \times \text{addresses per slot}) + 2$$

The first slot on each interface belongs to the node, so it is subtracted, and the plus two covers the system pods on the node’s own network.
Addresses per slot is the part people forget: 1 by default, 16 in prefix mode.

Step, on one instance with 3 interfaces and 6 slots each	Default	Prefix mode
Slots the instance offers	$3 \times 6 = 18$	$3 \times 6 = 18$
Minus the primary address, one per interface	$18 - 3 = 15$	$18 - 3 = 15$
Times the addresses one slot holds	$15 \times 1 = 15$	$15 \times 16 = 240$
Plus the 2 system pods on the host network, which need no address	$15 + 2 = 17$	$240 + 2 = 242$



Off by default. A setting on the networking addon, 1.9 or later, Nitro types only.



Needs contiguous /28s. A fragmented subnet can fail to hand out a prefix.



EKS clamps the total. 110 pods on smaller instances, 250 on large, so this node reports 110.

Addons: EKS-managed or self-managed

Pod networking, the service proxy and in-cluster DNS are what make a cluster usable. You choose at creation how they are run.



Pod networking

vpc-cni



Service proxy

kube-proxy



In-cluster DNS

coredns

Self-managed

the default at creation

- ✓ Bootstrapped as ordinary workloads inside the cluster.
- ✓ Full control of the manifests and the exact versions you run.
- ! You apply every upgrade yourself, and you carry the compatibility risk.

EKS-managed addons

opt in at create time

- ✓ EKS installs and version-manages each one through its own API.
- ✓ An upgrade is a version bump, applied with cluster-version awareness.
- ✓ Less maintenance, with guardrails on which versions fit your cluster.



EKS bootstraps self-managed versions unless you say otherwise, so turn that default off at create time and adopt them as managed addons. For a cluster kept simple and current, managed is the better default.

THE THREE ADDONS

Networking, service proxy, DNS

Each does one job. Miss any one and the cluster breaks in a specific, recognizable way.

Addon	Runs as	What it does	Missing it means
Pod networking <code>vpc-cni</code>	A pod on every node	Hands each pod a real IP address from your VPC subnets	Pods get no addresses, so nodes never report Ready
Service proxy <code>kube-proxy</code>	A pod on every node	Programs each node so Service traffic reaches a live pod	Traffic sent to a Service goes nowhere
DNS <code>coredns</code>	Ordinary pods, 2 replicas	Resolves Service and pod names to addresses in the cluster	Nothing in the cluster can resolve a name



The minimum, not the whole set

These three make a cluster usable. Production adds more, such as the EBS CSI driver so workloads can use persistent volumes.



DNS availability

EKS starts CoreDNS with two replicas and a disruption budget. Production raises the count and spreads it across nodes and zones.



Two ways to run them

A new cluster installs all three as plain manifests you update yourself. As EKS addons instead, you pick the version and AWS applies it.

